

Introduction to Programming with Scientific Applications (Spring 2025)

Final project

Study ID	Name	% contributed
202307989	Mikkel Moth Billing	100%

max 3 students

Briefly state the contributions of each of the group members to the project

I did all the coding and the report writing.

Declaration on Generative Artificial Intelligence (see <https://studerende.au.dk/en/gai>)

I used generative artificial intelligence (GAI) to complete this project. I used ChatGPT and Copilot to help with bugfixing and to get a better understanding of the processes of machine learning.

I have stated examples of what GAI have been used for in the report.

Note on plagiarism

Since the evaluation of the project report and code will be part of the final grade in the course, **plagiarism in your project handin will be considered cheating at the exam.** Whenever adopting code or text from elsewhere you must state this and give a reference/link to your source. It is perfectly fine to search information and adopt partial solutions from the internet – actually, this is encouraged – but always state your source in your handin. Also discussing your problems with your project with other students is perfectly fine, but remember each group should handin their own solution. If you are in doubt if your solution will be very similar to another group because you discussed the details, please put a remark that you have discussed your solution with other groups. For more Aarhus University information on plagiarism, please visit <http://library.au.dk/en/students/plagiarism/>.

1 Introduction

For this project I had to look at Image Classification, by creating a simple neural network. This neural network is used to classify images of handwritten digits from the MNIST database. These images are given in 28x28 grayscale images, where each pixel is a number between 0 and 255. I then use linear classification using a 784x10 matrix and a bias vector of size 10, to guess the number. A cost can be calculated from this guess, which is the mean square error of the guess and the actual number. By minimizing this cost function using linear algebra, the weight matrix, and bias vector, can be updated using back propagation. The weights can then be visualized and the cost and accuracy can be plotted, to show how the neural network evolves.

2 Code

Throughout the code I have used the decorator `@time_it`, that we have from the decorators lecture, to time the functions used.

2.1 Reading Data

For getting the raw data out of the MNIST files I had to first understand the format of the .gz files. For this I have used ChatGPT to help me understand how to read the bytes in the file, to get the magic numbers. As can be see at <https://github.com/sunsided/mnist> the first 4 bytes of the files are the magic number, which is 2049 for label files and 2051 for images files. Then the files contains 4 bytes decribing the size of the file, additionally the image files contains 8 bytes for the size the images. After that comes the raw data which is a byte per label or pixel.

To read the bytes I open the files with `gzip.open` then read the files with `.read`. Then I plot the images with their correct label, to see that the data have been read correctly, since the first 5 numbers of `t10k-labels.idx1-ubyte.gz` are [7, 2, 1, 0, 4] as seen in figure 1.

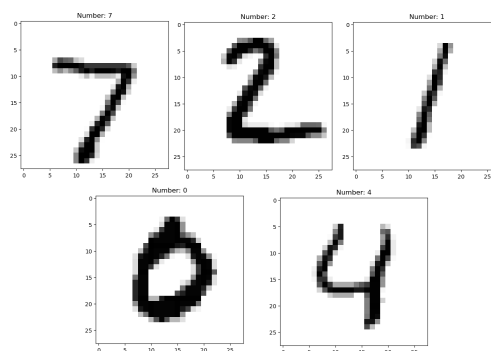


Figure 1: The first 5 images of the MNIST test set.

I also coded a way to read and write JSON files so that the weight matrix and bias vector can be saved. This is done with the json library. By doing this the weights and biases from MNIST can be visualized, as seen in figure 2.

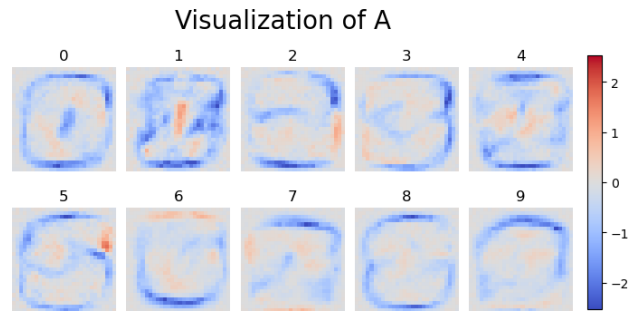


Figure 2: The weights of the MNIST neural network.

2.2 Linear Algebra

Since numpy is not allowed in the project, I had to write linear algebra functions to use on list vectors and matrices. These include vector addition, subtraction, and scalar multiplication. For these I wrote assertion tests to check if the given vectors are the same length, and that the scalar is actually a number. These tests are the main test that run throughout the code, and have helped if there were problems with the format of the weights matrix. I also wrote transpose and vector-matrix multiplication functions.

The linear algebra used states that the guess for a number is found by calculating:

$$a = xA + b,$$

where x is the image (in a 1×784 vector), A is the weight matrix (784×10) and b is the bias vector (1×10). With this the guess for the number is the index of the largest number in the vector a . From this guess the cost can be calculated as:

$$\text{cost}(a, y) = \sum_i \frac{(a_i - y_i)^2}{10}.$$

So I have written functions to calculate these values and find the max index.

For all the linear algebra functions I have used list comprehension as I find it easier to write and understand. The `evaluate` function also calculates, which calculates the predictions and cost, also calculates the accuracy, which is the number of correct guesses divided by the total number of guesses. I have then rewritten the `plot_images` function so it takes the optional argument `predictions`, which plots the images with their predicted labels.

2.3 Machine learning

To make the neural network, Stochastic Gradient Descent is used. This means that the given training images are split up into random batches, for this the `create_batches` function is used, which uses the `random` library to shuffle the

images. Then the network can be updated using formulas:

$$b_j -= \sigma \frac{1}{n} \sum_j 2 \frac{a_j - y_j}{10},$$

$$A_{ij} -= \sigma \frac{1}{n} \sum_{i,j} 2 x_i \frac{a_j - y_j}{10},$$

where σ is the step length which is set to 0.1. Notice the $-=$ because I did not, and i spend a lot of time not understanding why thing didn't work. Here I used ChatGPT to help me understand the problem, and it told me i was updating the weights and biases instead of subtracting the values. In the code i first calculate the derivative of the cost function using the linear algebra functions. I then use a nestet for loop to calculate every index of A .

After that is the actual training of the network in the `learn` function. Here I first create a uniformly random network, I then run it for an amount of epochs, where in each epoch the training set is split up into random batches of a set size. For every batch i run the `update` function, i then make a save the weights matrix and evaluation of the network, to be used in animations. For each epoch the time taken is recorded, and the network is evaluated on the full test set, and saved using `lineal_save` if it is more accurate then the last.

3 Evaluation

Along with these function I have written functions to plot and animate the evolution of the weights matrix, and the cost/accuracy of the network. In figure 3 the traines weights matrix is shown. We can see a similar style to the MNIST weights, exept here we have more possitive weights. The accuracy is calculated after each batch, and after 3rd batch the accuracy maxes out at 85.3% **cost**. We can also see the cost and accuracy as a function uf number of updates in figure 4.

The animations have been uploaded to brightspace and can be seen here [Here](#).

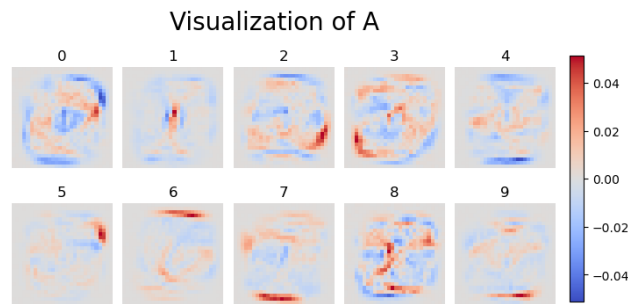


Figure 3: Visualization of the weights matrix at 5 epochs with batches of size 100.

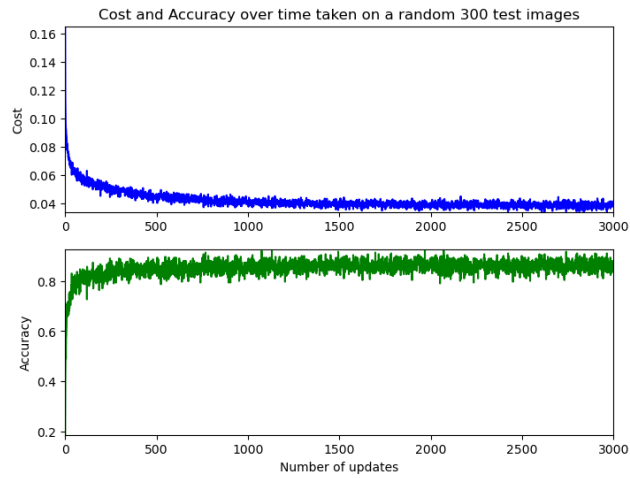


Figure 4: Cost and accuracy as a function of number of updates.

The time taken to learn the neural network is around 3 minutes, at a batch size of 100, per epoch (on laptop). To make the graphs I had to append a copy of every weight matrix to a list, and evaluate the network after every batch. This added a lot of execution time to the code, created an option to not save this data, and only evaluated on 300 random images. When running this on PC where it would normally take 1 minute per epoch, it instead takes 2.5 minutes and 6.5 minutes on laptop. I also tried to play with the batch size, and found that a size of 2000 would run the full function in 1 minute per epoch, but the accuracy only reaches around 83% at 5 epochs.

4 Conclusion